LAB 7: FUZZING FOR INPUT VALIDATION BUGS

Name: Vaishali Krishnan
USN: PES1UG24AM313

1. Objective

To identify and fix input validation bugs in a Python application using the Hypothesis fuzz testing library.

2. Tools Used

Python, Hypothesis, Pytest, Git Bash

3. Procedure

Step 1: Installing Dependencies

Installed the required libraries using:

python -m pip install hypothesis pytest

Step 2: Running the Original Application

Executed the original application using:

python processor.py

Provided valid inputs:

- Quota: 500MB
- Compression: 1024:256
- Tier: COLD

The application produced the expected output.

```
vaikr@vaishali-acer MINGW64 /c/PES/SE/git/SE_Lab/lab_7 (lab6-coverage)
$ python processor.py
==== Running parse_storage_quota ====
Enter quota (e.g. 500MB, 10GB, 2TB): 500MB
Quota: 0.50 GB

==== Running compute_compression_ratio ====
Enter raw:compressed bytes (e.g. 1024:256): 1024:256
Compression Ratio: 4.0

==== Running lookup_bandwidth_tier_rate ====
Enter tier descriptor (e.g. HOT_TIER, COLD, ARCHIVE): COLD
Rate per GB: $ 0.05
(.venv)
```

Figure 1: Original application output.

Step 3: Writing Hypothesis Tests

Implemented three tests in test_processor.py for:

1. parse_storage_quota()

2. compute_compression_ratio()
3. lookup_bandwidth_tier_rate()

Used Hypothesis strategies to generate random strings and None values to check whether the functions could handle unexpected inputs without crashing.

Step 4: Testing the Original Application

Executed:

python -m pytest test_processor.py

All three tests failed. Hypothesis identified errors caused by invalid inputs such as empty strings, None, and malformed values.

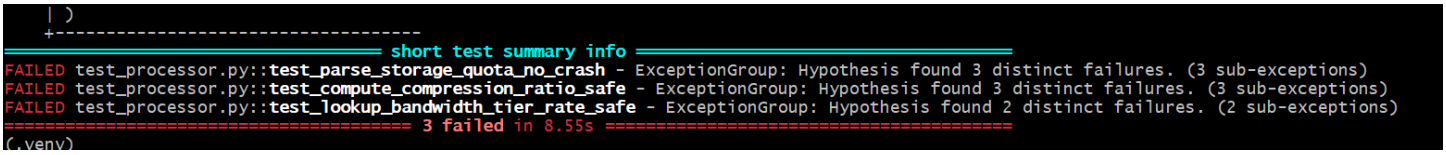


Figure 2: Initial fuzz testing failures.

Step 5: Fixing the Application

Modified processor.py by adding:

- Input type checking.
- Validation for empty and malformed inputs.
- Safe numeric conversions.
- Checks to prevent division by zero.
- Safe dictionary lookups.

These changes prevent invalid inputs from causing application crashes.

Executed the modified application again using:

python processor.py

The application produced the expected results for valid inputs.

```
vaikr@vaishali-acer MINGW64 /c/PES/SE/git/SE_Lab/lab_7 (lab6-coverage)
$ python processor.py
==== Running parse_storage_quota ====
Enter quota (e.g. 500MB, 10GB, 2TB): 500MB
Quota: 0.50 GB

==== Running compute_compression_ratio ====
Enter raw:compressed bytes (e.g. 1024:256): 1024:256
Compression Ratio: 4.0

==== Running lookup_bandwidth_tier_rate ====
Enter tier descriptor (e.g. HOT_TIER, COLD, ARCHIVE): COLD
Rate per GB: $ 0.05
(.venv)
vaikr@vaishali-acer MINGW64 /c/PES/SE/git/SE_Lab/lab_7 (lab6-coverage)
$
```

Figure 3: Fixed application output.

Step 6: Re-running the Tests

Executed the tests again:

python -m pytest test_processor.py

All three tests passed successfully.

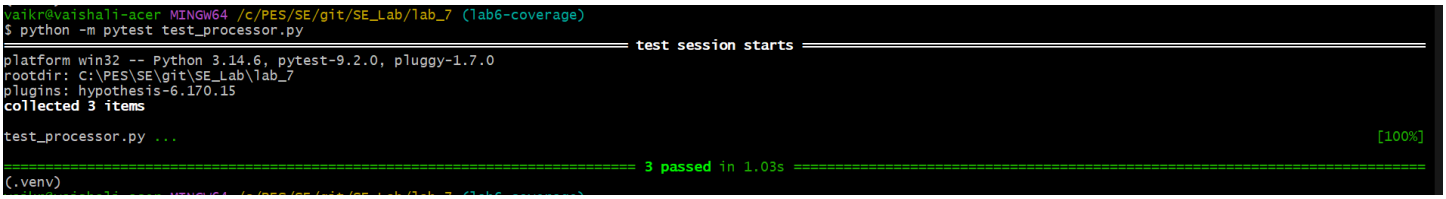


Figure 4: Successful fuzz testing with 3 passed tests.

4. Result

Fuzz testing was successfully performed using Hypothesis. Input validation bugs were identified and fixed in all three functions. The modified application passed all three fuzz tests.

5. Conclusion

This experiment demonstrated how fuzz testing can identify errors caused by unexpected inputs. Using Hypothesis and appropriate input validation improved the robustness and reliability of the Python application.